

WORDPRESS ACCESSIBILITY MEETUP — JUNE 2026

Presenter Guide

Building Your First Accessible Gutenberg Block

PRESENTER EDITION — includes full script, timing, and talking notes

Emilio Dominguez

Session Overview

Time	Topic	WCAG / Key Concepts
0–5 min	Intro & why blocks matter	The case — 1 in 6 people, 97% of sites fail
5–15 min	POUR framework + conformance levels	A / AA / AAA — target is AA
15–30 min	Environment setup — live demo	Node.js, LocalWP, npx create-block
30–50 min	File structure + adding attributes	block.json, edit.js, save.js
50–60 min	Compile, activate, test in browser	npm start, WP Admin, inserter
60–70 min	Accessibility layer — deep dive	ARIA, contrast, keyboard, screen reader
70–80 min	Q&A + wrap-up	Checklist + resources

Your goal for today

Every attendee leaves with a working block they built themselves — and understands every line. Start from what they know. Build one concept at a time. If something breaks, that's the lesson.

01 Intro — Why Blocks Matter

0–5 min

Opening Script

"Hi everyone, I'm Emilio. Today we're going to do something practical — by the end of this session you will have a real, working WordPress block plugin running on your machine. We're not going to talk theory for an hour. We're going to build."

■ Opening engagement — raise of hands

'Raise your hand if you've ever used a shortcode like [contact-form] or [gallery].'

→ Pause and let people raise hands.

'Blocks are the modern replacement for those. Instead of memorising shortcode syntax, users get a visual, drag-and-drop component. Our job today: build one of those.'

Key Numbers to Share

Stat	What It Means
1 in 6 people worldwide have a disability	Your block's HTML reaches real people who depend on it being accessible
97.4% of home pages have WCAG failures	Almost every site fails — building it right from scratch is a competitive advantage
WordPress powers 43% of the web	A well-built accessible block can touch millions of sites

■ Presenter Note

Keep this section short — 5 minutes max. The audience is here to BUILD, not listen.
The stats land harder if you pause after each one and let them sit.

02 POUR Framework + WCAG Levels

5–15 min

The POUR Framework — say this clearly

Principle	Plain English	Block-Relevant Example
P — Perceivable	Users can see/hear/feel all content	alt text, aria-label, colour contrast
O — Operable	Everything works without a mouse	Tab navigation, focus visible, no keyboard trap
U — Understandable	UI is predictable and clear	Clear labels, error messages, language attr
R — Robust	Works across current and future AT	Valid HTML, correct ARIA, semantic structure

Conformance Levels — we're building to AA

Level	Criteria	Legal Baseline?	Our Target?
A — Minimum	30 criteria	Rarely sufficient	Floor, not ceiling
AA — Standard	50 criteria (A + AA)	EU, UK, Australia, US ADA	✓ YES — build to this
AAA — Enhanced	78 criteria (all)	Aspire where possible	Bonus where easy

■ Talking Point — WCAG 2.2 (October 2023)

New in 2.2: 2.4.11 Focus Not Obscured (AA) — focused element can't be entirely hidden by sticky headers. We'll implement this in the CSS :focus-visible rule.

■ Presenter Note

Ask: 'Has anyone heard of WCAG before?' Most will have — let them anchor the discussion.
Don't go deep on every criterion. The goal is: they understand POUR and know AA is the target.

03 Environment Setup — Live Demo

15–30 min

■ Pre-Session Checklist — confirm these before going live

- Node.js installed: run `node -v` → should show v18 or higher
- npm installed: run `npm -v` → should show v9 or higher
- LocalWP open with a local site running (green dot)
- VS Code (or any editor) open
- Terminal / Command Prompt open and ready

Live Demo Script — type these commands on screen

```
# Step 1 – navigate to the WordPress plugins folder
$ cd ~/Local\ Sites/my-site/app/public/wp-content/plugins

# Step 2 – scaffold the block plugin (the magic command)
$ npx @wordpress/create-block my-notice-block

# Step 3 – enter the folder and start the compiler
$ cd my-notice-block
$ npm start
```

■ While npm installs (~60 seconds)

Talk: 'npm is downloading the entire WordPress JavaScript toolkit onto your machine.'

'This only happens once — after today you're ready for any block project.'

If it fails: run `npm cache clean --force` then retry.

■ Activate in WordPress after npm start is running

- Browser → wp-admin → Plugins → find 'My Notice Block' → click Activate
- Create a new Page → type / → search 'notice' → click to insert
- Show the block appearing in the editor — this is the 'wow' moment.

■ Presenter Note

Have your terminal VISIBLE on screen at all times during this section.

Ask the room to follow along. Give 2 minutes for people who are behind to catch up.

The block appearing in the editor is the moment people get excited — pause and let them see it.

04 File Structure + Adding Attributes

30–50 min

Walk Through Each File Before Coding

File	What It Does	What You'll Change
block.json	Block config — name, title, attributes, supports. WP reads first.	Add noticeText + noticeType attributes
src/edit.js	Editor view — what the admin sees in the dashboard.	Add RichText + InspectorControls
src/save.js	Frontend view — HTML saved to the database.	Add semantic section + aria attributes
src/index.js	Entry point — registers the block with WordPress.	No changes needed
src/style.scss	Shared styles — editor and frontend.	Add notice type colours + :focus-visible

■ The most important concept — say this clearly and pause

edit.js = what the ADMIN sees inside the WordPress dashboard
 save.js = what the VISITOR sees on the live website
 They CAN look completely different. That's intentional and powerful.
 Example: edit.js might show a colour picker. save.js just outputs the final coloured div.

Step 1 — Add attributes to block.json

```
"attributes": {
  "noticeText": {
    "type": "string",
    "default": "Write your notice here..."
  },
  "noticeType": {
    "type": "string",
    "default": "info"
  }
},
```

Step 2 — Update src/edit.js

```
import { useBlockProps, RichText, InspectorControls }
  from '@wordpress/block-editor';
import { PanelBody, SelectControl } from '@wordpress/components';

const ICONS = { info:'■', warning:'■', error:'X', success:'√' };

export default function Edit({ attributes, setAttributes }) {
  const { noticeText, noticeType } = attributes;
  const blockProps = useBlockProps({ className: `notice notice--${noticeType}` });
  return (
    <>
      <InspectorControls>
        <PanelBody title='Notice settings'>
          <SelectControl label='Notice type'
            value={noticeType}
            options={[{label:'Info',value:'info'},{label:'Warning',value:'warning'},
              {label:'Error',value:'error'},{label:'Success',value:'success'}}]
          />
        </PanelBody>
      </InspectorControls>
      <div {blockProps}>
        <div>{noticeText}</div>
      </div>
    </>
  );
}
```

```

        onChange={ (v) => setAttributes({ noticeType: v }) } />
      </PanelBody>
    </InspectorControls>
    <section {...blockProps}
      role="region" aria-label={` ${noticeType} notice`} >
      <span aria-hidden='true'>{ICONS[noticeType]}</span>
      <RichText tagName='p' value={noticeText}
        onChange={ (v) => setAttributes({ noticeText: v }) }
        placeholder='Write your notice here...' />
    </section>
  </>
);
}

```

Step 3 — Update src/save.js

```

import { useBlockProps, RichText } from '@wordpress/block-editor';

const ICONS = { info: '■', warning: '■', error: 'X', success: '√' };

export default function save({ attributes }) {
  const { noticeText, noticeType } = attributes;
  const blockProps = useBlockProps.save({ className: `notice notice--${noticeType}` });
  return (
    <section {...blockProps}
      role="region"
      aria-label={` ${noticeType} notice`}
      aria-live="polite"
    >
      <span aria-hidden='true'>{ICONS[noticeType]}</span>
      <RichText.Content tagName='p' value={noticeText} />
    </section>
  );
}

```

■ Presenter Note

Walk through the files on screen BEFORE typing any code.
 Open VS Code with the folder visible. Show `ls -la src/` in the terminal.
 Ask: 'Which file do you think controls what you see in the editor?' — let them guess.
 'npm start is watching your files. Every time you save, it recompiles automatically.'
 'Do a normal refresh (F5/Cmd+R) — not a hard refresh — to see the change.'
 'Red error in the editor? Section 9 of the handout covers every common error.'

05 Compile, Activate, Test

50–60 min

Checklist — do these together as a group

1. npm start is running in the terminal (shows 'watching for changes...')
2. Plugin is Activated in wp-admin → Plugins
3. Create a new test Page in WordPress
4. Type / in the editor → search 'notice' → click to insert
5. Type text into the block — it should update live
6. Click the block → check the right sidebar → change the Notice type dropdown
7. See the icon change in the editor (■ / ■ / ✕ / ✓)
8. Save the page → visit the frontend → confirm the notice appears

If the block shows a red 'unexpected content' error

This happens when save.js changes after content was already saved.

Fix: remove the block from the page, re-add it, re-type the content.

Prevent during development: always use a fresh test page.

06 Accessibility Deep Dive

60–70 min

ARIA Attributes in Your Block — explain each one

Attribute	WCAG Criterion	What It Does — say this to the room
<code>role="region"</code>	1.3.1, 4.1.2	Turns the wrapper into a named landmark. Screen reader users jump between landmarks with one keystroke.
<code>aria-label="..."</code>	4.1.2	Gives the landmark a readable name. Without it, <code>role=region</code> is ignored by most screen readers.
<code>aria-live="polite"</code>	4.1.3	If content changes dynamically, screen readers announce it when the user is idle — not mid-sentence.
<code>aria-hidden="true"</code>	1.1.1	The icon is already described by <code>aria-label</code> . Hiding it prevents duplicate announcements.
<code><section></code> not <code><div></code>	1.3.1	A <code><div></code> means nothing to AT. A <code><section></code> says 'self-contained content'. Semantic = accessible.
<code>:focus-visible</code> CSS	2.4.7, 2.4.11	Keyboard users must always see which element is focused. NEVER write <code>*:focus { outline: none }</code> .

Colour Contrast — Required Ratios

WCAG Criterion	Ratio	Applies To
1.4.3 Normal text (AA)	4.5 : 1	Text under 18pt or 14pt bold
1.4.3 Large text (AA)	3 : 1	18pt+ or 14pt bold+
1.4.11 Non-text contrast (AA)	3 : 1	Buttons, icons, input borders
2.4.11 Focus Not Obscured (2.2)	3 : 1	Focus ring itself must have sufficient contrast

■ Colour Independence — 1.4.1

Never use colour as the ONLY way to convey the notice type.

Your block already handles this: colour + icon + `aria-label` all carry the meaning.

Live demo: the warning notice has ■ + yellow + `aria-label='warning notice'`. Three signals, not one.

Quick Testing Checklist

- Put your mouse away — tab through the block. Can you reach everything?
- Run axe DevTools (free Chrome/Firefox extension) — should show zero errors
- Test with NVDA + Firefox (Windows) or VoiceOver + Safari (Mac)
- Check colour contrast at webaim.org/resources/contrastchecker
- Verify `:focus-visible` shows a visible outline on the dismiss button

■ Presenter Note

Live demo: open axe DevTools on the page with your block. Show zero violations.

If you have time: turn on VoiceOver (Cmd+F5 on Mac) and tab through the block live.

The audience finds hearing 'info notice region' announced by the screen reader very impactful.

07 Wrap-Up + Q&A

70–80 min

Summary to read out

1. You scaffolded a block plugin with one command: `npx @wordpress/create-block`
2. You understand `edit.js` (admin view) vs `save.js` (visitor view)
3. You added attributes so users can change text and notice type without touching code
4. You output accessible HTML: `role`, `aria-label`, `aria-live`, semantic `<section>`
5. You know how to compile, activate, and test your block
6. You have a WCAG 2.1 AA foundation — not bolted on, built in from line one

■ Resources to Share with the Room

- Block Editor Handbook: developer.wordpress.org/block-editor/
- `@wordpress/create-block`: developer.wordpress.org/block-editor/reference-guides/packages/packages-create-block/
- WordPress Accessibility Team: make.wordpress.org/accessibility/
- WCAG 2.1 Quick Reference: www.w3.org/WAI/WCAG21/quickref/
- axe DevTools (free): deque.com/axe/devtools/
- WebAIM Contrast Checker: webaim.org/resources/contrastchecker
- WP Accessibility Meetup: equalizedigital.com/wordpress-accessibility-meetup/

■ If You Run Out of Time

Skip the accessibility deep-dive details — the handout covers everything.

The most important outcome: everyone has a working block they built themselves.

That confidence is the deliverable. Code details can be looked up.

■ Presenter Note

Leave 10 minutes for Q&A. The best questions come from people who tried it live.
Point to the handout for anything you didn't get to — it has all the code.